Kubernetes Chapter 07

Deploymentで APIサーバーを安定稼働させる

ReplicaSetの自動管理とロールバックで高可用性を実現する

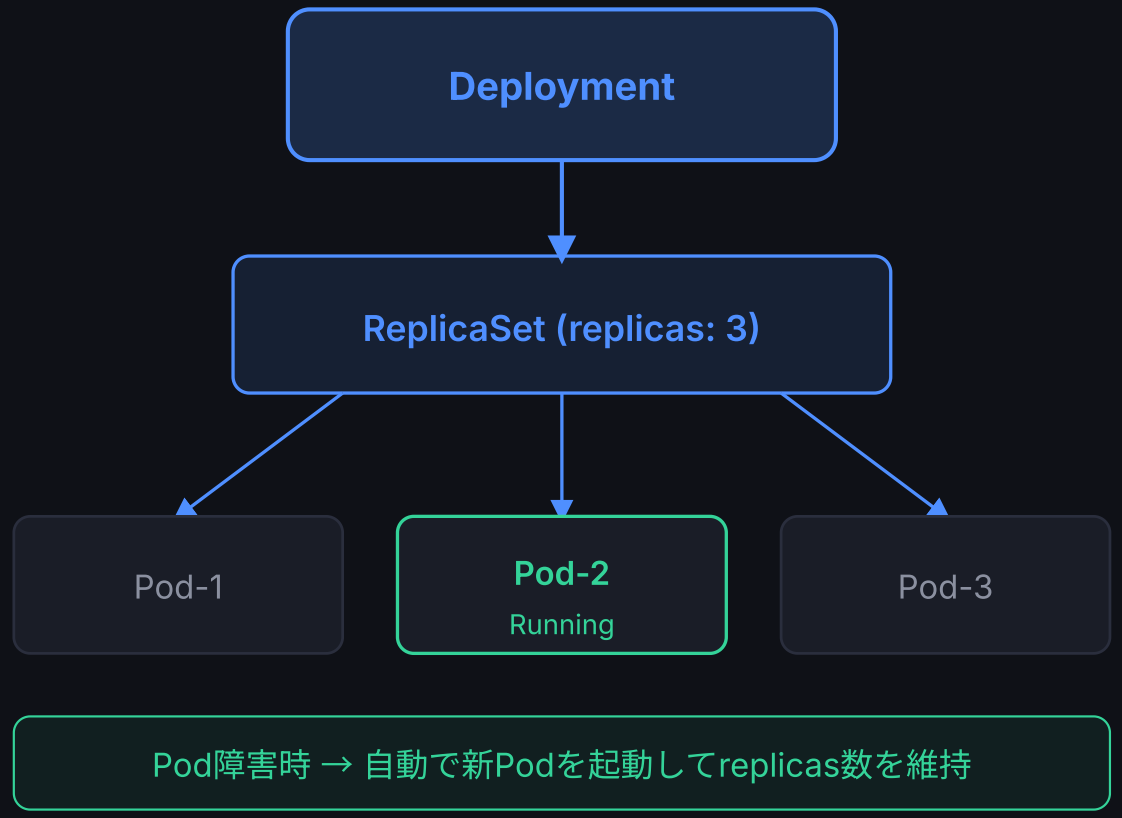
viserhaut — 2026

SECTION 01

DeploymentとReplicaSet

DeploymentはECSサービスに相当するPod管理リソース

- **ECSサービス**がタスク定義から複数タスクを起動・維持するように、Deploymentは指定したreplicas数のPodを常に稼働させる
- Pod障害時は**自動で新しいPodを起動**し、replicas数を維持する
- 裏側でReplicaSetを自動生成。Deploymentはそのライフサイクルを管理するラッパー



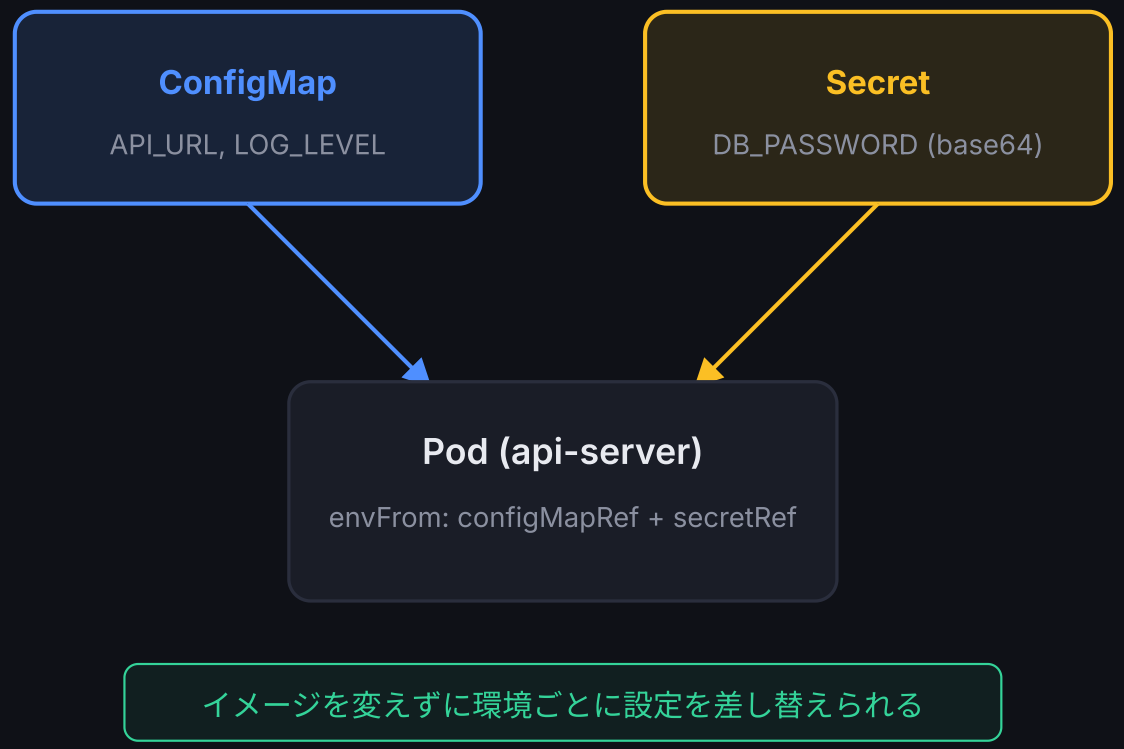
ReplicaSet直接操作 vs Deployment — なぜDeploymentを使うか

観点	REPLICASET（直接）	DEPLOYMENT
ローリング更新	手動でPodを差し替える必要あり	自動でローリング更新を実行する
ロールバック	旧ReplicaSetを手動で復元	kubectl rollout undo で1コマンド
更新履歴	追跡できない	revisionHistoryLimit で世代管理
推奨度	直接操作は非推奨	本番運用では常にDeploymentを使う

SECTION 02

ConfigMap・SecretとAPI構築

ConfigMap + SecretでAPIの環境変数を外部注入



- ConfigMapはAPI_URL・LOG_LEVELなど非機密の設定値を管理する
- SecretはDB_PASSWORD・API_KEYなど機密値を管理。base64エンコードで保存
- どちらも**envFrom**でPodに一括注入できる。イメージを変えずに環境差し替えが可能

Deployment + ConfigMap + Secret マニフェスト

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: api-server:v1.0
          envFrom:
            - configMapRef:
                name: api-config
            - secretRef:
                name: api-secret
```

replicas: 3
常に3Pod稼働を維持。障害時は自動で新Podを起動

selector.matchLabels
ReplicaSetがどのPodを管理するかをラベルで特定

envFrom
ConfigMapとSecretを両方一括で環境変数に注入

ロールアウトとロールバック — kubectl rollout

- **イメージを更新**するだけで自動ローリングアップデート。旧Podを順番に置き換えることで無停止デプロイを実現
- 問題が発生したら `kubectl rollout undo` で**即座に前バージョンに戻せる**
- **rollout status**でデプロイ進捗をリアルタイム確認。CI/CDパイプラインから呼び出すことも多い

```
# 新イメージでデプロイ
kubectl set image deploy/api api=api-server:v2.0

# ロールバック
kubectl rollout undo deploy/api

# 進捗確認
```

まとめ

01

DeploymentはECSサービス相当

ReplicaSetを自動生成し、replicas数のPodを常に維持する。Pod障害時は自動で新Podを起動する

02

ConfigMap + Secretで設定分離

非機密値はConfigMap、機密値はSecretで管理しenvFromで注入する。イメージを変えずに環境を差し替えられる

03

rollout undoで即ロールバック

Deploymentは更新履歴を保持するため、問題発生時に1コマンドで前バージョンに戻せる